



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



**TFG del Grado en Ingeniería
Informática**

**Cloud Computing: Análisis,
Diseño y Despliegue de un
Servicio Empresarial Seguro
en Microsoft Azure**



Presentado por Karima Drafi Rico
en Universidad de Burgos — Febrero 2026
Tutor: D. José Ignacio Santos Martín



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



D. José Ignacio Santos Martín, profesor del departamento de Ing. de Organización, área de OE.

Expone:

Que el alumno Dña. Karima Drafli Rico, con DNI 72748765h, ha realizado el Trabajo final de Grado en Ingeniería Informática titulado “Cloud Computing: Análisis, Diseño y Despliegue de un Servicio Empresarial Seguro en Microsoft Azure”.

Y que dicho trabajo ha sido realizado por el alumno bajo la dirección de los que suscriben, en virtud de lo cual se autoriza su presentación y defensa.

En Burgos, Febrero de 2026

Vº. Bº. del Tutor:

Vº. Bº. del co-tutor:

D. José Ignacio Santos Martín

D. José Ignacio Santos Martín

Resumen

En este Trabajo Fin de Grado se presenta un análisis, diseño y despliegue de un servicio empresarial seguro utilizando Microsoft Azure. Se describen la arquitectura cloud adoptada, los servicios implementados y las prácticas de seguridad y control de versiones aplicadas durante el desarrollo del proyecto.

Descriptores

Cloud Computing, Microsoft Azure, arquitectura basada en servicios, seguridad informática, integración continua, despliegue de aplicaciones.

Abstract

This Final Degree Project presents the analysis, design, and deployment of a secure enterprise service using Microsoft Azure. It describes the adopted cloud architecture, implemented services, and the security and version control practices applied during the project development.

Keywords

Cloud Computing, Microsoft Azure, service-based architecture, information security, continuous integration, application deployment.

Índice general

Índice general	iii
Índice de figuras	v
Índice de tablas	vi
1. Introducción	1
2. Objetivos del proyecto	3
3. Conceptos teóricos	5
3.1. Cloud Computing	5
3.2. Microsoft Azure	5
3.3. Arquitectura basada en servicios	6
3.4. API REST	6
3.5. DevOps	6
4. Técnicas y herramientas	9
4.1. Metodología de desarrollo	9
4.2. Lenguaje y framework backend	9
4.3. Control de versiones	9
4.4. Plataforma cloud	9
4.5. Integración continua	10
4.6. Calidad del código	10
4.7. Terraform	11
5. Aspectos relevantes del desarrollo del proyecto	13
5.1. Ciclo de vida del proyecto	13

5.2. Arquitectura del sistema	13
5.3. API REST implementada	14
5.4. Componente de inteligencia artificial	15
5.5. Integración continua y despliegue	15
5.6. Automatización con Logic App	15
5.7. Seguridad y gestión de secretos	16
5.8. Validación del prototipo	16
6. Trabajos relacionados	17
7. Conclusiones y Líneas de trabajo futuras	19
Bibliografía	21

Índice de figuras

Índice de tablas

6.1. Comparativa de trabajos y soluciones relacionadas	18
--	----

1. Introducción

En los últimos años, las tecnologías cloud se han convertido en una pieza fundamental dentro de las infraestructuras empresariales modernas. Las organizaciones necesitan aplicaciones seguras, escalables y accesibles desde cualquier ubicación, permitiendo gestionar información crítica de forma eficiente y reduciendo la complejidad de mantenimiento de infraestructuras físicas.

Uno de los problemas más frecuentes en entornos empresariales es la gestión de solicitudes internas de tecnología, como peticiones de acceso, cambios de configuración, creación de entornos, altas de aplicaciones o incidencias técnicas. Muchas organizaciones continúan utilizando herramientas poco integradas como hojas de cálculo, cadenas de correos electrónicos o sistemas internos difíciles de mantener. Esto provoca problemas de trazabilidad, retrasos en la resolución y dificultades para monitorizar el estado global del servicio.

El presente Trabajo Fin de Grado propone el análisis, diseño e implementación de un sistema empresarial seguro basado en tecnologías cloud utilizando Microsoft Azure. La solución desarrollada permite registrar solicitudes operativas TI, clasificarlas automáticamente según su prioridad y tipo, generar recomendaciones operativas y gestionar la información de forma segura utilizando servicios cloud gestionados.

La arquitectura propuesta utiliza una API desarrollada en Python y Flask desplegada sobre Azure App Service, integrando además servicios como Azure Storage, Azure Key Vault, Azure Logic Apps y herramientas de automatización y calidad como GitHub Actions y SonarCloud.

Durante el desarrollo del proyecto se han aplicado metodologías ágiles basadas en sprints, utilizando herramientas profesionales de gestión de tareas y control de versiones para garantizar la trazabilidad del trabajo realizado.

La memoria se estructura en los siguientes capítulos:

- Capítulo 1: Introducción y contexto del proyecto.
- Capítulo 2: Objetivos generales y específicos.
- Capítulo 3: Conceptos teóricos relacionados con cloud computing y arquitecturas empresariales.
- Capítulo 4: Técnicas y herramientas utilizadas.
- Capítulo 5: Aspectos relevantes del desarrollo.
- Capítulo 6: Trabajos relacionados.
- Capítulo 7: Conclusiones y líneas futuras.

Finalmente, se incluyen diferentes anexos técnicos relacionados con la planificación, diseño, requisitos, documentación técnica y manuales de usuario.

2. Objetivos del proyecto

El objetivo principal del proyecto es desarrollar una plataforma cloud segura en Microsoft Azure capaz de automatizar la gestión de solicitudes operativas TI mediante una arquitectura moderna basada en servicios gestionados.

Para alcanzar este objetivo principal se han definido los siguientes objetivos específicos:

- Analizar las ventajas de las plataformas cloud en entornos empresariales.
- Diseñar una arquitectura segura basada en servicios cloud.
- Implementar una API REST utilizando Python y Flask.
- Desplegar la aplicación en Microsoft Azure.
- Gestionar secretos y configuraciones sensibles mediante Azure Key Vault.
- Implementar mecanismos de integración continua utilizando GitHub Actions.
- Gestionar el proyecto utilizando metodologías ágiles y control de versiones.
- Diseñar un flujo automatizado basado en eventos que permita procesar solicitudes operativas mediante Azure Logic Apps.
- Implementar una clasificación automática de solicitudes utilizando un componente de inteligencia artificial ligera.

- Analizar la calidad del código utilizando SonarQube/SonarCloud integrado con GitHub Actions.
- Generar documentación técnica y funcional del sistema.

Además de los objetivos técnicos, el proyecto busca aplicar conocimientos adquiridos durante el Grado en Ingeniería Informática relacionados con ingeniería del software, seguridad informática, arquitecturas cloud y desarrollo backend.

3. Conceptos teóricos

3.1. Cloud Computing

El cloud computing es un modelo tecnológico que permite acceder a recursos informáticos bajo demanda utilizando Internet. Este modelo elimina la necesidad de mantener infraestructuras físicas propias y permite escalar recursos dinámicamente [3].

Entre las principales ventajas del cloud computing destacan:

- Escalabilidad.
- Alta disponibilidad.
- Pago por uso.
- Reducción de costes.
- Facilidad de mantenimiento.

3.2. Microsoft Azure

Microsoft Azure es la plataforma cloud de Microsoft orientada al desarrollo, despliegue y administración de aplicaciones empresariales [1].

Los principales servicios utilizados en este proyecto son:

- Azure App Service.
- Azure Storage.

- Azure Key Vault.
- Azure Monitor.
- Application Insights.

3.3. Arquitectura basada en servicios

Las arquitecturas basadas en servicios permiten dividir una aplicación en componentes independientes encargados de responsabilidades concretas.

Este enfoque facilita:

- Escalabilidad.
- Mantenimiento.
- Reutilización.
- Seguridad.

3.4. API REST

Una API REST permite la comunicación entre sistemas mediante peticiones HTTP.

En este proyecto se utilizan diferentes endpoints para:

- Crear solicitudes operativas.
- Consultar solicitudes e incidencias.
- Obtener métricas.

3.5. DevOps

DevOps es una metodología orientada a integrar desarrollo y operaciones con el objetivo de automatizar procesos y mejorar la calidad del software [2].

Entre las prácticas utilizadas destacan:

- Integración continua.

- Despliegue automatizado.
- Control de versiones.
- Automatización de procesos.

4. Técnicas y herramientas

4.1. Metodología de desarrollo

El proyecto se ha desarrollado siguiendo principios de metodologías ágiles utilizando iteraciones organizadas en sprints.

Para la gestión de tareas se ha utilizado Zube.io, permitiendo organizar historias de usuario, tareas técnicas y seguimiento del progreso.

4.2. Lenguaje y framework backend

La aplicación backend se ha desarrollado utilizando Python junto con el framework Flask.

Flask permite construir APIs REST ligeras y fáciles de mantener.

4.3. Control de versiones

El control de versiones se ha realizado utilizando Git y GitHub.

Se han realizado commits frecuentes para mantener trazabilidad completa del desarrollo.

4.4. Plataforma cloud

La infraestructura cloud se basa en Microsoft Azure y en servicios gestionados documentados por Microsoft para aplicaciones web, almacenamiento, monitorización y gestión de secretos [7, 8, 4].

Los servicios principales utilizados son:

- Azure App Service.
- Azure Storage.
- Azure Key Vault.
- Azure Monitor.

4.5. Integración continua

Se ha configurado un pipeline CI/CD utilizando GitHub Actions, siguiendo el enfoque de despliegue automatizado de aplicaciones en Azure App Service [5].

Este sistema automatiza:

- Validación del código.
- Preparación del despliegue.
- Automatización de tareas.

4.6. Calidad del código

La calidad del código se ha preparado para analizarse mediante SonarQube/SonarCloud integrado con GitHub Actions [9].

En este proyecto se configura SonarCloud como servicio gestionado compatible con repositorios GitHub. Su uso permite obtener métricas objetivas de mantenibilidad, fiabilidad y seguridad sin desplegar un servidor SonarQube propio. El análisis está preparado para ejecutarse desde el pipeline de integración continua y complementar a las pruebas unitarias.

Esta herramienta permite detectar:

- Vulnerabilidades.
- Duplicidad.
- Complejidad excesiva.

- Posibles errores.
- Deuda técnica y problemas de mantenibilidad.

Los resultados del análisis deberán emplearse como evidencia de calidad del código y como apoyo para priorizar mejoras antes de la entrega final.

4.7. Terraform

Terraform es una herramienta Infrastructure as Code (IaC) utilizada para automatizar el aprovisionamiento de recursos cloud.

En el proyecto se emplea para definir recursos Azure de forma declarativa, permitiendo despliegues reproducibles, controlados y versionados mediante Git [6].

5. Aspectos relevantes del desarrollo del proyecto

5.1. Ciclo de vida del proyecto

El proyecto se ha organizado en cinco sprints siguiendo una metodología ágil.

- Sprint 1: Análisis del problema empresarial, definición de requisitos y arquitectura inicial.
- Sprint 2: Provisionamiento de infraestructura Azure y configuración de Terraform.
- Sprint 3: Desarrollo de la API Flask, portal web y persistencia en Azure Storage.
- Sprint 4: Seguridad con Key Vault, Managed Identity y automatización con Logic App.
- Sprint 5: Pruebas, CI/CD, documentación y validación del prototipo.

5.2. Arquitectura del sistema

La arquitectura implementada sigue un modelo de servicios gestionados en Microsoft Azure. Los componentes principales son:

- **Azure App Service:** hospeda la API Flask y el portal web empresarial en Python 3.11.

- **Azure Storage:** persiste las solicitudes operativas TI en un contenedor Blob privado.
- **Azure Key Vault:** almacena el token de autenticación de la API.
- **Azure Logic App:** orquesta el flujo automatizado de solicitudes mediante un trigger HTTP.
- **GitHub Actions:** ejecuta pruebas, validación Terraform y despliegue continuo.

El flujo principal del sistema es el siguiente:

1. Un empleado envía una solicitud operativa TI desde el portal web o mediante la Logic App.
2. Azure App Service procesa la petición con Gunicorn.
3. La API valida los datos y clasifica automáticamente la solicitud.
4. El sistema genera una recomendación operativa para el equipo de TI.
5. Los datos se almacenan en Azure Blob Storage.
6. Los secretos se obtienen desde Azure Key Vault mediante Managed Identity.
7. El endpoint de métricas expone el estado agregado del sistema.

5.3. API REST implementada

La API desarrollada en `src/app.py` expone los siguientes endpoints:

- `GET /`: estado del servicio y versión.
- `GET /portal`: portal web para solicitudes operativas TI.
- `GET /health`: comprobación de salud y modo de almacenamiento.
- `POST /solicitudes`: creación de solicitudes con clasificación automática.
- `GET /solicitudes`: listado con filtros por estado, prioridad y tipo.

- GET /metricas: métricas agregadas del sistema.

Al crear una solicitud, el módulo `classifier.py` analiza el título y la descripción para asignar tipo de solicitud (*acceso, entorno, aplicación, configuración, incidencia*), prioridad (*baja, media, alta*), categoría (*seguridad, infraestructura, soporte, general*) y una recomendación operativa, devolviendo un valor de confianza asociado.

5.4. Componente de inteligencia artificial

Se ha incorporado un clasificador ligero basado en análisis semántico de palabras clave y reglas de negocio. Este componente cumple el objetivo del TFG de aportar análisis automático de información operativa sin depender de infraestructuras corporativas complejas. El clasificador evalúa términos relacionados con seguridad, infraestructura y soporte para determinar la categoría, prioridad y recomendación de cada solicitud operativa TI.

5.5. Integración continua y despliegue

Se ha implementado un pipeline en GitHub Actions (`.github/workflows/deploy.yml`) que:

1. Ejecuta pruebas unitarias de la API.
2. Valida y aplica la infraestructura con Terraform.
3. Despliega el código Python en Azure App Service.

5.6. Automatización con Logic App

La Logic App `logic-tfg-solicitudes-dev` recibe solicitudes operativas TI mediante un trigger HTTP, las reenvía a la API para su clasificación y construye una notificación operativa. Este componente demuestra la automatización de procesos empresariales sin intervención manual directa sobre la API.

5.7. Seguridad y gestión de secretos

La autenticación de la API utiliza un token Bearer almacenado en Azure Key Vault. La App Service accede al secreto mediante Managed Identity, evitando credenciales embebidas en el código fuente. El almacenamiento de solicitudes reside en un contenedor Blob con acceso privado.

5.8. Validación del prototipo

Las pruebas unitarias en `tests/test_api.py` verifican la creación de solicitudes, el listado, las métricas, el portal web y la clasificación automática. El prototipo puede ejecutarse localmente con almacenamiento en fichero JSON o desplegarse en Azure con persistencia en Blob Storage.

6. Trabajos relacionados

Actualmente existen numerosas soluciones empresariales orientadas a la gestión de incidencias, solicitudes internas y servicios cloud.

Herramientas como Jira Service Management, ServiceNow o Zendesk ofrecen funcionalidades avanzadas para la gestión de incidencias empresariales.

Sin embargo, estas plataformas suelen implicar costes elevados y arquitecturas complejas.

En el ámbito académico también existen diferentes trabajos relacionados con arquitecturas cloud desplegadas en plataformas como Microsoft Azure o Amazon Web Services.

El presente proyecto se diferencia por combinar:

- Arquitectura cloud.
- Seguridad empresarial.
- Automatización DevOps.
- Gestión de incidencias.
- Integración continua.

Además, el sistema desarrollado busca mantener una arquitectura sencilla y fácilmente escalable.

Frente a estas herramientas, el proyecto no busca competir funcionalmente con plataformas comerciales, sino demostrar el diseño e implementación

Solución	Enfoque	Ventajas	Limitaciones
ServiceNow	ITSM empresarial	Muy completo y configurable	Coste y complejidad altos
Jira Service Management	Gestión de tickets	Integración con desarrollo ágil	Requiere configuración avanzada
Zendesk	Soporte y atención	Interfaz madura y rápida adopción	Menor control sobre infraestructura
Proyecto desarrollado	Solicitudes TI en Azure	Código propio, cloud e IaC	Alcance académico y prototipo

Tabla 6.1: Comparativa de trabajos y soluciones relacionadas

de una solución cloud segura, automatizada y verificable dentro del alcance de un Trabajo Fin de Grado.

7. Conclusiones y Líneas de trabajo futuras

El desarrollo de este Trabajo Fin de Grado ha permitido aplicar conocimientos relacionados con ingeniería del software, arquitecturas cloud, seguridad informática y automatización de procesos.

Uno de los principales resultados obtenidos ha sido el diseño e implementación de una plataforma empresarial segura utilizando Microsoft Azure para la gestión y automatización de solicitudes operativas TI.

La solución desarrollada demuestra cómo es posible construir aplicaciones empresariales modernas utilizando servicios cloud gestionados y herramientas DevOps.

La solución implementada demuestra cómo los servicios cloud gestionados permiten automatizar procesos empresariales reduciendo la intervención manual y mejorando la trazabilidad de las operaciones.

Durante el proyecto se han utilizado tecnologías empleadas en entornos empresariales reales: GitHub Actions, Terraform, Azure App Service, Azure Blob Storage, Azure Key Vault y Azure Logic Apps.

Además, el proyecto ha permitido profundizar en aspectos relacionados con:

- Seguridad cloud.
- Gestión de secretos.
- APIs REST.

- Automatización de procesos.
- Arquitecturas escalables.
- Clasificación automática de información operativa.

El prototipo desarrollado valida los objetivos del TFG: automatización de un proceso empresarial (gestión de solicitudes operativas TI), seguridad mediante Key Vault y Managed Identity, despliegue automatizado con CI/CD, e incorporación de un componente de análisis automático para clasificación y recomendación operativa.

Como líneas futuras se plantean:

- Integrar Azure OpenAI para enriquecer la clasificación y las recomendaciones.
- Incorporar autenticación de usuarios con Azure AD.
- Añadir Application Insights y dashboards de monitorización.
- Ampliar el portal web con seguimiento de solicitudes para usuarios no técnicos.
- Migrar la persistencia a Azure SQL o Cosmos DB para consultas más complejas.
- Conectar conectores de correo en Logic App para notificaciones automáticas.

Bibliografía

- [1] Azure Architecture Center, Microsoft Learn. Browse azure architectures, 2025. Consultado: marzo 2026.
- [2] Karthik Bojja. Scalable devops practices with azure: Automating secure kubernetes deployments. *IJIRSET*, 9(6), Junio 2020.
- [3] Microsoft. Microsoft azure: Cloud computing dictionary, 2025. Consultado: marzo 2026.
- [4] Microsoft Learn. Azure monitor, 2025.
- [5] Microsoft Learn. Deploy to azure app service by using github actions, 2025. Consultado: marzo 2026.
- [6] Microsoft Learn. Terraform for azure, 2025.
- [7] Microsoft Learn. Tutorial: Access storage from a web app with managed identities, 2025. Consultado: marzo 2026.
- [8] Microsoft Learn. Tutorial: Use a managed identity to connect key vault to an azure web app in .net, 2025. Consultado: marzo 2026.
- [9] SonarSource. Sonarqube documentation, 2025.